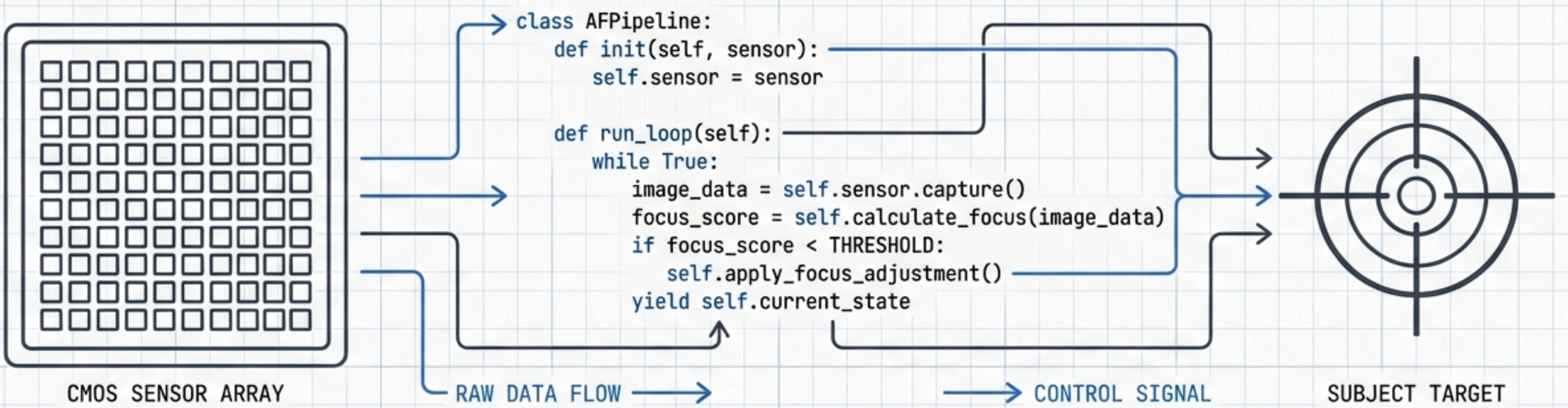


MOBILE COMPUTATIONAL PHOTOGRAPHY // CHAPTER 5

Autofocus System Implementation

A Practical Guide to Building a Python-Based AF Pipeline

From Theory to Code: Engineering the Feedback Loop

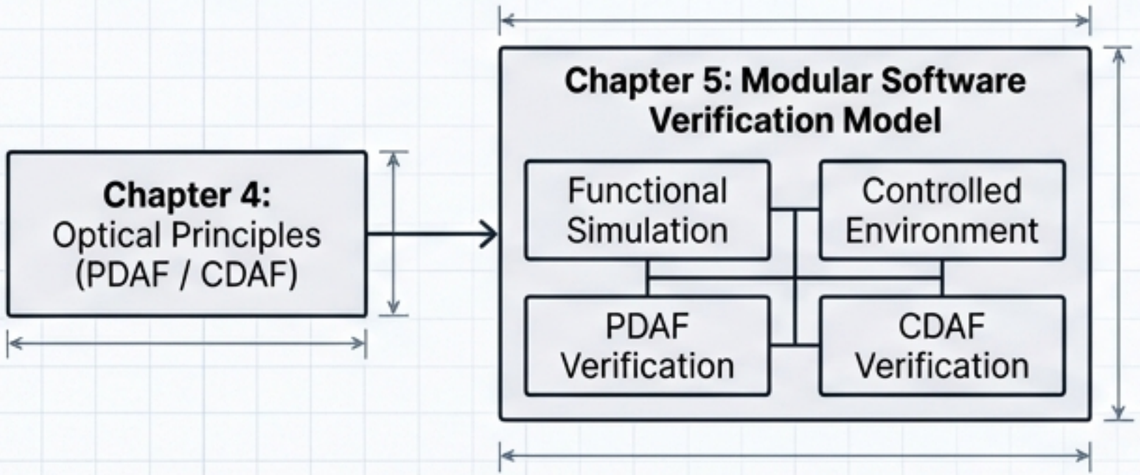
NotebookLM

Overview & Objectives

Bridging Theory and Application

The Context

Chapter 5 transitions from the optical principles of Autofocus (Chapter 4) into a modular software verification model. We are building a functional simulation to verify PDAF and CDAF theories in a controlled environment.



The Objectives List

- **Environment Setup:** Establishing a Python-based simulation lab.
- **Architecture:** Understanding the modular separation of Data, Logic, and Control.
- **Data Handling:** Parsing complex RAW sensor data (Bayer vs. PD).
- **Execution:** Configuring algorithms and evaluating convergence.

Goal: Build a functional software verification model to simulate real-world lens behavior.

NotebookLM

Quick Start: Environment Setup

Prerequisites for the AF Simulation

- [ ] **Dependencies**
  Ensure Python 3.x, NumPy, SciPy, Matplotlib.
- [ ] **Installation**
  Clone repository and install requirements.
- [ ] **Verification**
  Run component tests to confirm stability.

Note: The simulation treats image files as "sensor input" to test algorithms without physical hardware.

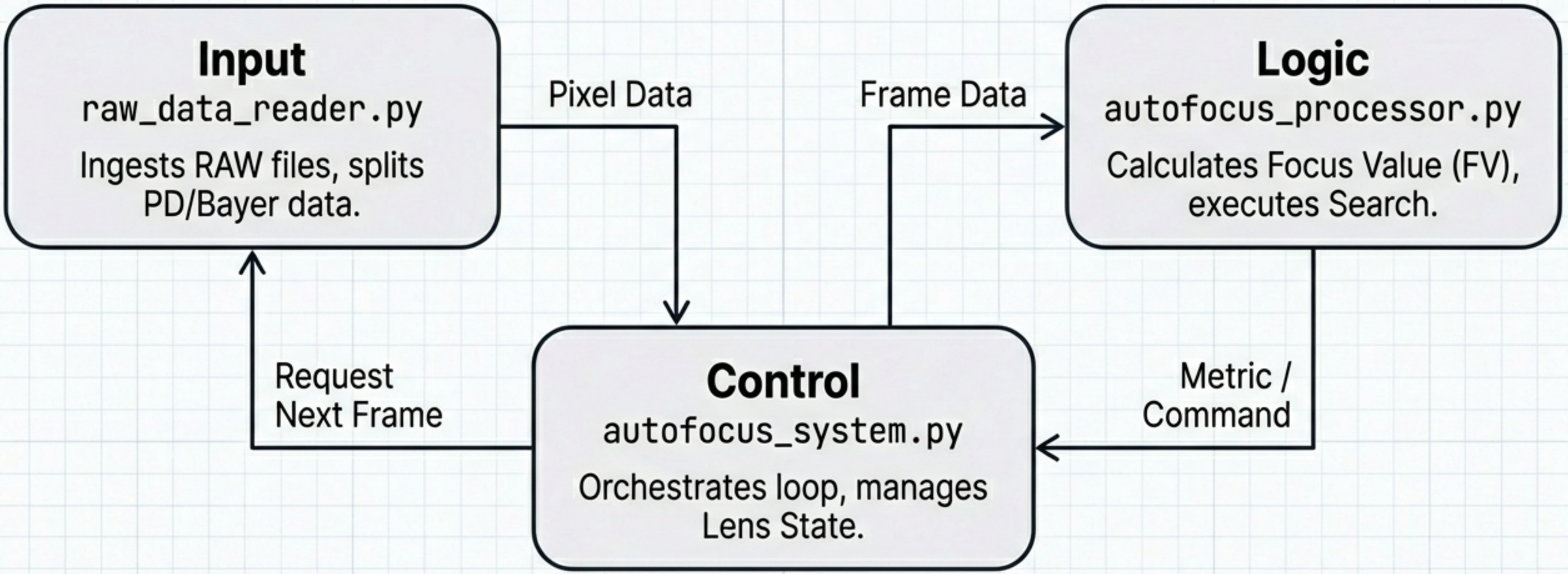
bash — 80×24

```
$ pip install numpy scipy matplotlib
[Success] Libraries installed.
$ git clone https://github.com/course/af-pipeline.git
$ cd af-pipeline
$ python run_tests.py
[OK] Reader Module passed.
[OK] Processor Module passed.
[OK] System Module passed.
$ _
```

NotebookLM

System Architecture

The Modular Design Pattern



Design Philosophy: Strict separation of concerns. Hardware abstraction (Reader), Math (Processor), and Orchestration (System).

NotebookLM

Core Component 1: Raw Data Reader

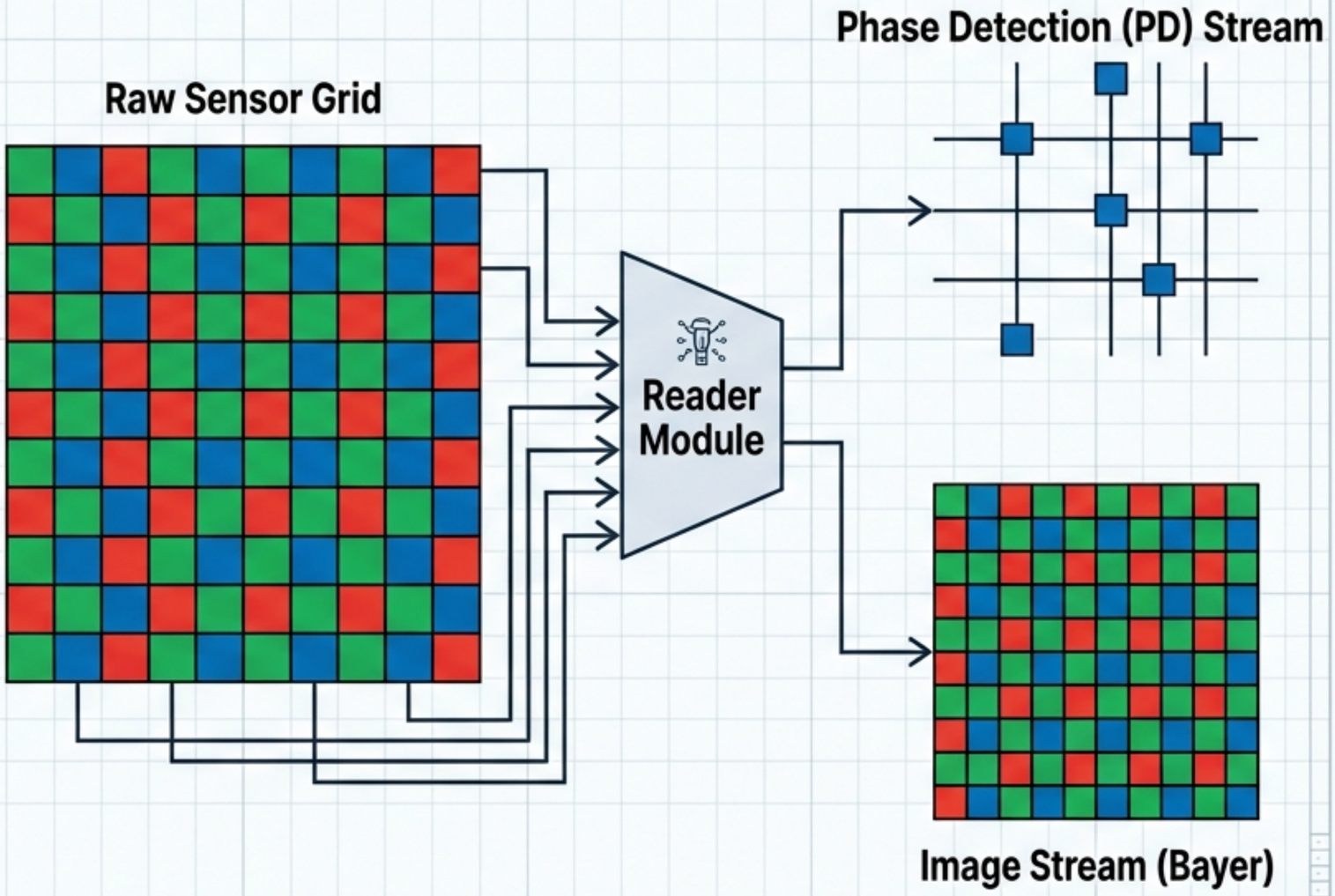
File: `raw_data_reader.py`

**Role:

Abstracts hardware complexity and parses binary sensor data.

**Key Functions:

- **Ingestion:** Reads raw binary files.
- **Pattern Extraction:** Identifies Bayer configuration (e.g., RGGB, BGGR).
- **Pixel Separation:** Critical function—isolates Phase Detection pixels used for depth from the standard image pixels.



NotebookLM

Core Component 2: Autofocus Processor

File: `autofocus_processor.py`

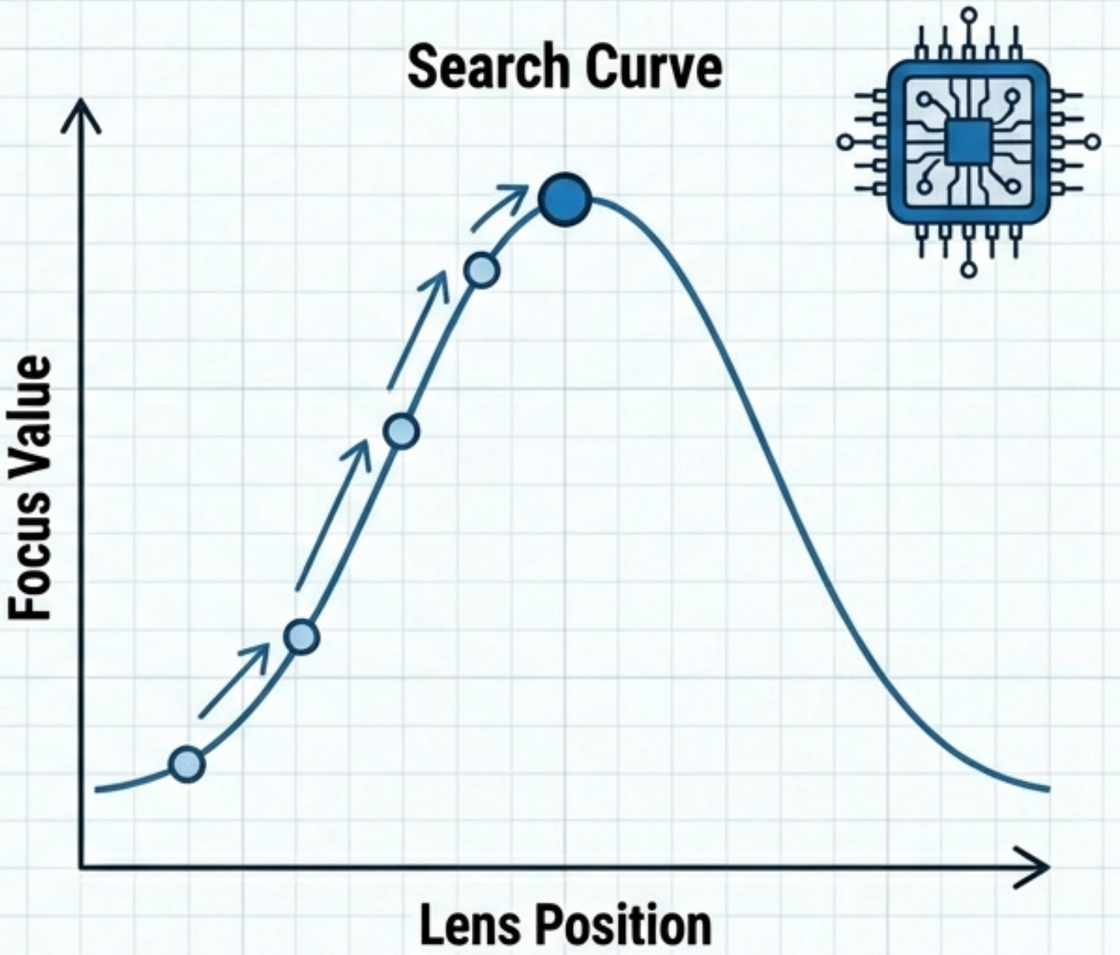
**Role:

The mathematical engine converting pixels into control metrics.

**Key Responsibilities:

- **Metric Calculation:** Computes Focus Value (FV) or sharpness scores for Contrast Detection.
- **Depth Estimation:** Processes PD data to calculate phase disparity (distance to subject).
- **Search Algorithm:** Executes logic to find the peak (e.g., Hill Climbing, Coarse-to-Fine).

This module contains the pure math and logic, independent of hardware state.



NotebookLM

Core Component 3: The Core AF System

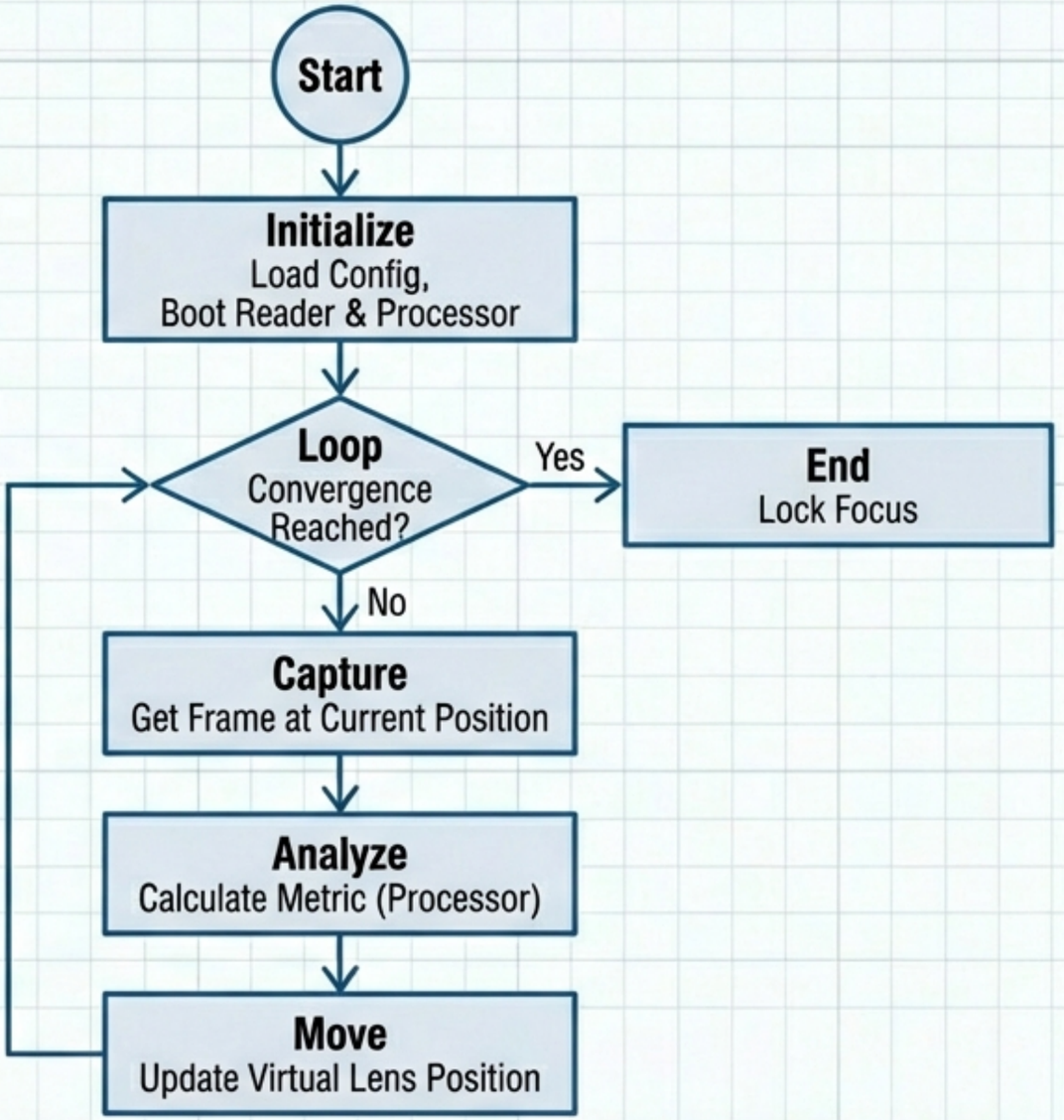
File: `autofocus_system.py`

**Role:

The Conductor. Integrates all components into a cohesive workflow.

**Functions:

State management, Initialization, and Feedback Loop execution.

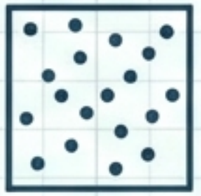


NotebookLM

The Dual Data Pipeline

Processing Real Data Streams

Phase Detection (PD) Path



Input:
Sparse PD Pixels

Processing: Phase Correlation / Disparity Calc.

Outcome: Speed & Direction. Calculates magnitude of lens movement to get ‘close’ quickly.

Role: Coarse Focus.

Bayer Contrast Path

Input:
Full RGB Bayer Data



Processing: High-Pass Filter / Contrast Metric.

Outcome: Precision & Peaking. Verifies sharpness peak.

Role: Fine Focus / Verification.

The system runs these paths sequentially or in parallel to balance Speed (PD) with Accuracy (Contrast).

NotebookLM

Key System Features

Capabilities of the Chapter 5 Codebase

Dual-Mode Processing

Native support for both Contrast-based (CDAF) and Phase-based (PDAF) algorithms within the same pipeline.

Modular Architecture

Algorithms can be swapped (e.g., changing '`Hill Climbing`' to '`Binary Search`') without rewriting the data loader.

Robust Error Handling

Compito manage sailures

Built-in management for bad sensor data, missing files, or read errors ensures system stability.

Visualization Tools

Visualization Tools

Utilities to plot Focus Value curves and visualize convergence paths for debugging and tuning.

NotebookLM

Configuration Strategy

Tuning the Engine via `config.json`

```
{
  "sensor": {
    "width": 4032,
    "height": 3024,
    "pattern": "RGGB",
    "bit_depth": 10
  },
  "autofocus": {
    "roi": [1500, 1200, 500, 500],
    "step_size": 10,
    "threshold": 0.05
  }
}
```

Data Parameters: Defines hardware specs (Resolution, Bit Depth, Bayer Pattern).

Region of Interest: Coordinates `[x, y, w, h]` for the focus area.

Search Strategy: Defines granularity of lens movement (Coarse vs. Fine).

External configuration allows adaptation to different sensors without code changes.

NotebookLM

Developer's View: Basic Usage

Running the System in Python

```python
from af_system import AutofocusSystem
from config import load_config

# 1. Initialize System with Config
config = load_config('settings.json')
af_system = AutofocusSystem(config)

# 2. Run the Autofocus Loop
print("Starting AF Sequence...")
result = af_system.run_autofocus()

# 3. Output the Result
if result.converged:
    print(f"Focus Locked at Lens Pos: {result.position}")
    print(f"Final Sharpness Score: {result.metric:.4f}")
else:
    print("Focus Failed to Converge.")
```

Instantiation

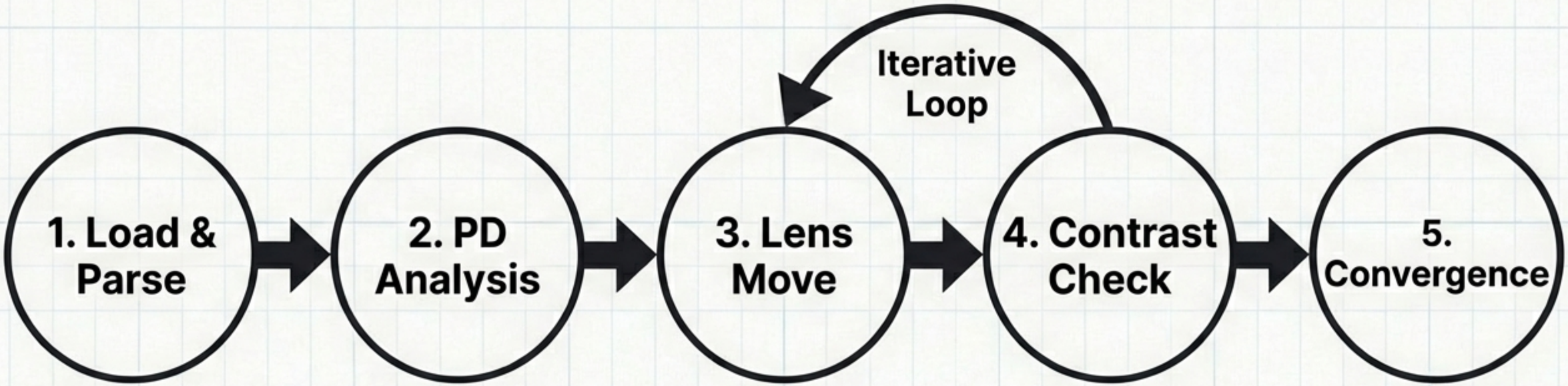
Trigger Main Loop

Review Metrics

NotebookLM

Real Data Processing Workflow

From Raw File to Focused Image



Extract ROI from RAW.

Compute Depth Estimate (Rough).

Jump to Estimated Position.

Analyze Bayer Contrast (Fine).

Lock if Peak Found.

JetBrains Mono
Inter

NotebookLM

Performance & Quality Metrics

How we measure success

Processing Efficiency

Steps to Converge

Number of iterations required to find focus. Lower is faster/better.

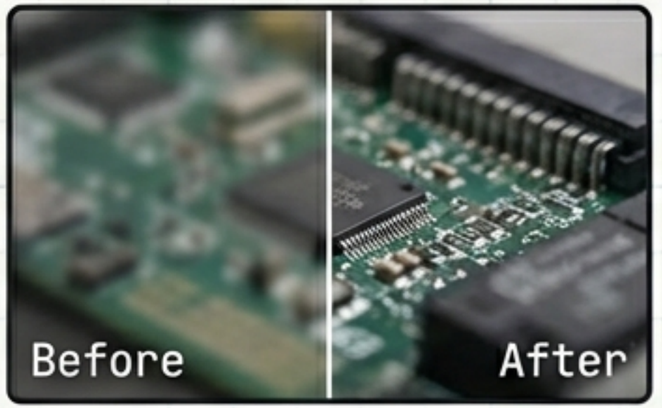
Quality Metric

Sharpness Score

Magnitude of high-frequency content in ROI. Higher is sharper.

$$\text{Metric} = \sum (\text{Gradient_X}^2 + \text{Gradient_Y}^2)$$

Visual Verification

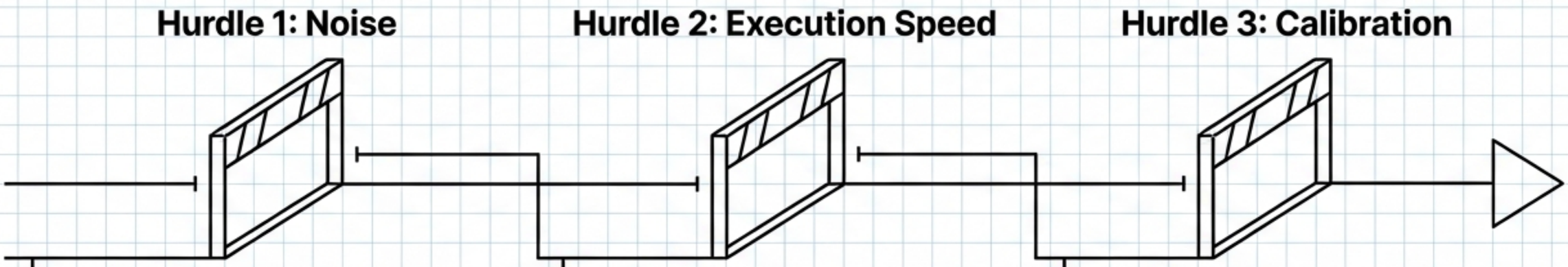


Perceptual check of the final output image.

NotebookLM

Challenges & Optimization

Moving from Simulation to Production



Hurdle 1: Noise

High ISO noise creates false "sharpness". Requires pre-processing `denoising`.

Hurdle 2: Execution Speed

Python is for prototyping. Production `ISPs` use `C++` or `FPGA/ASIC` hardware.

Hurdle 3: Calibration

`PDAF` relies on precise `Gain Maps`. Simulation assumes ideal sensors; reality requires calibration.

The Python model validates logic, but hardware implementation requires handling these physical realities.

NotebookLM

Chapter 5 Summary

Verification Model Complete



- **Achievement:** Built a modular software verification model for `Autofocus`.
- **Architecture:** Validated separation of `Reader`, `Processor`, and `System`.
- **Pipeline:** Successfully integrated mixed `PD` (`Depth`) and `Bayer` (`Contrast`) data.
- **Status:** System is configurable and ready for algorithm stress-testing.

→ Next: **Chapter 6: Auto White Balance** - Ensuring Color Accuracy.

NotebookLM